Frequency Encoding

1. Definition

Frequency encoding, also called *count encoding*, is a technique used to convert a categorical feature into a numerical one by replacing each category with its frequency of occurrence in the dataset. Instead of creating several binary variables as in one-hot encoding, frequency encoding produces a single numerical feature.

If a category c appears $count(c)$ times in a dataset of size n, then one may encode it either by its count

$$\mathrm{enc}(c) = count(c),$$

or by its proportion

$$\mathrm{enc}(c) = \frac{count(c)}{n}.$$

In practice, proportion-based encoding is often preferred, since it is more stable across datasets of different sizes.

2. Why Use Frequency Encoding?

Frequency encoding is useful for several reasons:

Dimensionality reduction: it transforms a categorical variable into a single numerical column, even when the number of categories is very large;

Simplicity: it is easy to compute and interpret;

High-cardinality handling: it is particularly suitable when a feature contains many distinct categories;

No target leakage: unlike target encoding, it only uses the feature itself and not the response variable;

Information on category prevalence: common categories receive larger values, while rare categories receive smaller ones.

3. Mathematical Formulation

Let $X = \{x_1, \ldots, x_n\}$ be a categorical feature taking values in the set of categories $\{c_1, \ldots, c_k\}$. The count-based frequency encoding of an observation x_i is

$$f(x_i) = \sum_{j=1}^{n} \mathbf{1}_{\{x_j = x_i\}},$$

where $\mathbf{1}_{\{\cdot\}}$ denotes the indicator function.

The proportion-based version is

$$f(x_i) = \frac{1}{n} \sum_{j=1}^{n} \mathbf{1}_{\{x_j = x_i\}}.$$

Thus, all observations belonging to the same category receive the same encoded value.

4. Worked Example

Consider the categorical feature

$$[\text{New York},\ \text{Los Angeles},\ \text{Chicago},\ \text{New York},\ \text{Houston},\ \text{Los Angeles},\ \text{New York},\ \text{Chicago}].$$

The frequencies are:

$$\text{New York} : 3, \qquad \text{Los Angeles} : 2, \qquad \text{Chicago} : 2, \qquad \text{Houston} : 1.$$

Since the dataset contains $n = 8$ observations, after the proportion-based encoding the data becomes:

$$[0.375,\ 0.25,\ 0.25,\ 0.375,\ 0.125,\ 0.35,\ 0.375,\ 0.25].$$

5. Advantages and Limitations

Frequency encoding has several advantages:

- it is computationally cheap; - it avoids the curse of dimensionality induced by one-hot encoding; - it can be effective when the frequency of a category is itself informative.

However, it also has important limitations:

- different categories with the same frequency become indistinguishable; - it does not preserve category identity; - the encoded value may be weakly related to the prediction target; - frequencies may change from one dataset to another, which can reduce stability.

6. Handling Unseen Categories

When a new category appears in test data but was absent in training data, one must define a default value. Common choices include:

- assigning 0; - assigning the minimum observed frequency; - assigning a small constant $\varepsilon > 0$.

The choice depends on whether unseen categories should be treated as extremely rare or simply unknown.

7. Comparison with Other Encodings

One-hot encoding. One-hot encoding creates one binary column per category. It preserves category identity, but may lead to very high dimensionality when the number of categories is large.

Label encoding. Label encoding assigns arbitrary integers such as $0, 1, 2, \ldots$ to categories. This is simple, but it may introduce an artificial ordering with no real meaning.

Frequency encoding. Frequency encoding avoids high dimensionality and gives a value with an interpretable meaning, namely the prevalence of the category. However, categories with identical frequencies receive the same representation.

8. Practical Variants

Several variants are often used in practice:

Log-frequency encoding:

$$f(c) = \log(1 + count(c)),$$

which compresses the range when some categories are extremely common;

Rare-category grouping: categories with very small frequencies are grouped into a single category such as "Other";

Time-aware frequency encoding: in temporal problems, frequencies are computed using only past data in order to avoid data leakage.

9. When to Use Frequency Encoding

Frequency encoding is particularly useful when: the feature has high cardinality; the popularity or rarity of a category is informative; memory and computational efficiency are important; one wants a simple encoding with no use of the target variable.

It is less appropriate when category identity matters strongly, or when many categories have similar frequencies.

10. Python Code

Below is a simple implementation of proportion-based frequency encoding in Python:

```python
from collections import Counter

def frequency_encoding(values):
    if not values:
        return []

    counts = Counter(values)
    n = len(values)

    return [counts[x] / n for x in values]
```